Spring Security & JWT Implementation Guide - CivilControl Backend

# Overview

This document explains the complete implementation of Spring Security with JWT (JSON Web Tokens) authentication in the CivilControl Backend system, providing secure access control, role-based authorization, and stateless authentication.

---

# Architecture

## Core Components

1. **SecurityConfig** - Main security configuration class
2. **JwtAuthenticationFilter** - Filter to validate JWT tokens
3. **JwtService** - Utility for generating and validating JWT tokens
4. **CustomUserDetailsService** - Loads user details for authentication
5. **AuthService** - Handles login, token generation, and refresh
6. **Permission & Role System** - Fine-grained access control

---

# Security Configuration

## SecurityConfig.java

Located in: src/main/java/CivilControl/backEnd/config/SecurityConfig.java
**Purpose**: Configures HTTP security, CORS, CSRF, and authentication filters.
**Key Features**:

- **Stateless Sessions**: No server-side session storage
- **JWT-based Authentication**: Token validation on every request
- **CORS Configuration**: Cross-origin requests allowed
- **Public Endpoints**: Login and documentation accessible without authentication
- **Role-based Authorization**: Endpoints protected by permissions

**Configuration Pattern**:

@Configuration @EnableWebSecurity @EnableMethodSecurity public class SecurityConfig { @Bean public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception { http .csrf(csrf -> csrf.disable()) .cors(cors -> cors.configurationSource(corsConfigurationSource())) .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)) .authorizeHttpRequests(auth -> auth .requestMatchers("/api/v1/auth/**").permitAll() .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll() .anyRequest().authenticated() ) .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class); return

http.build(); } }

---

# JWT Implementation

## 1. JwtService.java

Located in: src/main/java/CivilControl/backEnd/service/implementation/JwtService.java
**Purpose**: Generates, validates, and extracts information from JWT tokens.
**Key Methods**:
**A) Generate Access Token**:

public String generateAccessToken(User user) { Map<String, Object> claims = new HashMap<>(); claims.put("userId", user.getId()); claims.put("roles", user.getRoles().stream() .map(Role::getName) .collect(Collectors.toList())); return Jwts.builder() .setClaims(claims) .setSubject(user.getUsername()) .setIssuedAt(new Date()) .setExpiration(new Date(System.currentTimeMillis() + ACCESS_TOKEN_EXPIRATION)) .signWith(getSigningKey(), SignatureAlgorithm.HS256) .compact(); }
**B) Generate Refresh Token**:

public String generateRefreshToken(User user) { return Jwts.builder() .setSubject(user.getUsername()) .setIssuedAt(new Date()) .setExpiration(new Date(System.currentTimeMillis() + REFRESH_TOKEN_EXPIRATION)) .signWith(getSigningKey(), SignatureAlgorithm.HS256) .compact(); }
**C) Validate Token**:

public boolean isTokenValid(String token, UserDetails userDetails) { final String username = extractUsername(token); return (username.equals(userDetails.getUsername()) && !isTokenExpired(token)); }
**Token Configuration**:

- **Access Token Expiration**: 1 hour (configurable)
- **Refresh Token Expiration**: 7 days (configurable)
- **Algorithm**: HS256 (HMAC with SHA-256)
- **Secret Key**: Stored in application.properties

## 2. JwtAuthenticationFilter.java

Located in: src/main/java/CivilControl/backEnd/config/security/JwtAuthenticationFilter.java
**Purpose**: Intercepts HTTP requests to validate JWT tokens and set security context.
**Flow**:

1. Extract JWT token from Authorization header
2. Validate token format and signature
3. Extract username from token
4. Load user details from database
5. Validate token against user details
6. Set authentication in SecurityContext

7. Continue filter chain

**Implementation**:

```
@Override protected void doFilterInternal( HttpServletRequest request, HttpServletResponse response, FilterChain filterChain) throws ServletException, IOException { final String authHeader = request.getHeader("Authorization"); if (authHeader == null || !authHeader.startsWith("Bearer ")) { filterChain.doFilter(request, response); return; } final String jwt = authHeader.substring(7); final String username = jwtService.extractUsername(jwt); if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) { UserDetails userDetails = customUserDetailsService.loadUserByUsername(username); if (jwtService.isTokenValid(jwt, userDetails)) { UsernamePasswordAuthenticationToken authToken = new UsernamePasswordAuthenticationToken( userDetails, null, userDetails.getAuthorities()); authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request)); SecurityContextHolder.getContext().setAuthentication(authToken); } } filterChain.doFilter(request, response); }
```

## 3. AuthService.java

Located in: src/main/java/CivilControl/backEnd/service/implementation/AuthService.java

**Purpose**: Handles authentication operations (login, token refresh, logout).

**Key Operations**:

**A) Login**:

```
public LoginResponseDTO login(LoginRequestDTO request) { Authentication authentication = authenticationManager.authenticate( new UsernamePasswordAuthenticationToken( request.username(), request.password() ) ); User user = userRepository.findByUsername(request.username()) .orElseThrow(() -> new InvalidCredentialsException(...)); String accessToken = jwtService.generateAccessToken(user); String refreshToken = jwtService.generateRefreshToken(user); return new LoginResponseDTO(accessToken, refreshToken, user.getUsername()); }
```

**B) Refresh Token**:

```
public RefreshTokenResponseDTO refreshToken(String refreshToken) { String username = jwtService.extractUsername(refreshToken); User user = userRepository.findByUsername(username) .orElseThrow(() -> new InvalidTokenException(...)); if (!jwtService.isTokenValid(refreshToken, user)) { throw new InvalidTokenException(...); } String newAccessToken = jwtService.generateAccessToken(user); return new RefreshTokenResponseDTO(newAccessToken); }
```

---

# Permission & Role System

## 1. Permission Entity

**Attributes**:

- id: Unique identifier

- name: Permission name (e.g., “VEHICLE_READ”, “USER_CREATE”)
- description: Human-readable description
- module: General module (services, documents, vehicles, etc.)
- translatedName: Spanish translation for frontend display

**Example Permissions**:

VEHICLE_READ = "Leer Vehiculos" VEHICLE_CREATE = "Crear Vehiculos" USER_MANAGE = "Gestionar Usuarios" ROLE_MANAGE = "Gestionar Roles"

## 2. Role Entity

**Attributes**:

- id: Unique identifier
- name: Role name (e.g., “ADMIN”, “MANAGER”, “OPERATOR”)
- description: Role description
- permissions: Set of permissions
- active: Whether role is active
- deleted: Soft delete flag

**System Roles**:

- **ROOT**: Developer role (cannot be deleted or assigned)
- **ADMIN**: Full system access
- **MANAGER**: Department management
- **OPERATOR**: Basic operations

## 3. Method-Level Security

**Using @PreAuthorize**:

@PreAuthorize("hasAuthority('VEHICLE_READ')") @GetMapping("/{id}") public ResponseEntity<VehicleResponseDTO> getVehicle(@PathVariable Long id) { return ResponseEntity.ok(vehicleService.getVehicleById(id)); } @PreAuthorize("hasAuthority('VEHICLE_CREATE')") @PostMapping public ResponseEntity<VehicleResponseDTO> createVehicle(@RequestBody VehicleDTO dto) { return ResponseEntity.status(HttpStatus.CREATED) .body(vehicleService.createVehicle(dto)); }

---

# Authentication Flow

## 1. Login Flow

Client Server Database | | | |-- POST /api/v1/auth/login ---------------------------->| | { username, password } | | | | | |-- Authenticate User ------->| | |<-- User Details ------------| | | | | |-- Generate Access Token --->| | |-- Generate Refresh Token -->| | | | |<-- 200 OK --------------| | | { accessToken, refreshToken, username } |

### 2. Authenticated Request Flow

Client JwtFilter SecurityContext Service | | | | |-- GET /api/v1/vehicles (Bearer token) ----------->| | | | | | | |-- Validate Token ------>| | | |-- Load User ----------->| | | |-- Set Authentication -->| | | | | | | | |-- Check Permission -->| | | |<-- Response ---------| | | | | |<-- 200 OK (with data) --| | |

### 3. Refresh Token Flow

Client Server Database | | | |-- POST /api/v1/auth/refresh ------------------------->| | { refreshToken } | | | | | |-- Validate Refresh Token -->| | |<-- User Details ------------| | | | | |-- Generate New Access Token | | | | |<-- 200 OK --------------| | | { accessToken } |

---

# Configuration

## application.properties

# JWT Configuration jwt.secret=your-secret-key-here-should-be-at-least-256-bits-long jwt.expiration=3600000 # 1 hour in milliseconds jwt.refresh-expiration=604800000 # 7 days in milliseconds # Security spring.security.user.name=admin spring.security.user.password=admin

## Environment Variables (Production)

[![](http://localhost:63342/markdownPreview/2139562436/commandRunner/runrun.png)](http://localhost:63342/markdownPreview/2139562436/markdown-preview-index-ggi4h3hlbr5j509698nr27pdpg.html#)export JWT_SECRET=your-production-secret-key export JWT_EXPIRATION=3600000 export JWT_REFRESH_EXPIRATION=604800000

---

# API Endpoints

## Authentication Endpoints

#### 1. Login

POST /api/v1/auth/login Content-Type: application/json { "username": "admin", "password": "password123" } Response 200 OK: { "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...", "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...", "username": "admin" }

**2. Refresh Token**

POST /api/v1/auth/refresh Content-Type: application/json { "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..." } Response 200 OK: { "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..." }

**3. Validate Token**

GET /api/v1/auth/validate Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... Response 200 OK: { "valid": true }

### Protected Endpoints

**Example: Get All Vehicles**

GET /api/v1/vehicles Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... Response 200 OK: { "content": [...], "totalPages": 5, "totalElements": 50 } Response 401 Unauthorized (no token): { "status": 401, "message": "Token no proporcionado", "timestamp": "2026-01-20T15:30:00" } Response 403 Forbidden (no permission): { "status": 403, "message": "No tiene permisos para acceder a este recurso", "timestamp": "2026-01-20T15:30:00" }

---

# Best Practices

## 1. Token Storage (Frontend)

**Recommended**: Store in memory or sessionStorage

// Good sessionStorage.setItem('accessToken', token); // Avoid localStorage.setItem('accessToken', token); // Vulnerable to XSS document.cookie = `token=${token}`; // Vulnerable to CSRF

## 2. Token Refresh Strategy

**Implement automatic token refresh**:

// Intercept 401 responses and refresh token axios.interceptors.response.use( response => response, async error => { if (error.response?.status === 401) { const refreshToken = sessionStorage.getItem('refreshToken'); const newToken = await refreshAccessToken(refreshToken); sessionStorage.setItem('accessToken', newToken); return axios(error.config); } return Promise.reject(error); } );

## 3. Secure Password Handling

**Always use BCrypt**:

@Bean public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }

## 4. Permission Naming Convention

**Pattern**: ENTITY_ACTION
Examples:

- VEHICLE_READ
- VEHICLE_CREATE
- VEHICLE_UPDATE
- VEHICLE_DELETE
- USER_MANAGE
- ROLE_MANAGE

## 5. Logout Handling

**Client-side logout**:

function logout() { sessionStorage.removeItem('accessToken'); sessionStorage.removeItem('refreshToken'); window.location.href = '/login'; }
**Optional: Server-side token blacklist** (for enhanced security)

---

# Security Considerations

## 1. CORS Configuration

**Allow specific origins only**:

@Bean public CorsConfigurationSource corsConfigurationSource() { CorsConfiguration configuration = new CorsConfiguration(); configuration.setAllowedOrigins(Arrays.asList( "http://localhost:3000", // Development "https://yourdomain.com" // Production )); configuration.setAllowedMethods(Arrays.asList("GET", "POST", "PUT", "DELETE", "PATCH")); configuration.setAllowedHeaders(Arrays.asList("*")); configuration.setAllowCredentials(true); UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource(); source.registerCorsConfiguration("/**", configuration); return source; }

## 2. HTTPS in Production

**Always use HTTPS in production**:

# Force HTTPS server.ssl.enabled=true security.require-ssl=true

## 3. Token Secret Management

**Never commit secrets to version control**:

# BAD - Don't do this jwt.secret=mysecretkey123 # GOOD - Use environment variables

jwt.secret=${JWT_SECRET}

### 4. Rate Limiting

**Implement rate limiting for authentication endpoints** to prevent brute force attacks.

### 5. User Account Security

**Features implemented**:
- Password encryption (BCrypt)
- Soft delete for users
- Active/inactive user status
- Role-based access control

---

# Troubleshooting

## Issue: 401 Unauthorized even with valid token

**Solutions**:
1. Check token expiration
2. Verify Authorization header format: Bearer {token}
3. Ensure user exists and is active
4. Check token signature with correct secret

## Issue: 403 Forbidden

**Solutions**:
1. Verify user has required permission
2. Check role assignment
3. Ensure permission is correctly configured

## Issue: Token not being validated

**Solutions**:
1. Verify JwtAuthenticationFilter is registered
2. Check filter order in SecurityConfig
3. Ensure JwtService is properly configured

## Issue: CORS errors

**Solutions**:
1. Add frontend URL to allowed origins
2. Enable credentials if needed
3. Check allowed methods and headers

---

# Testing

## Manual Testing with Postman/Insomnia

**1. Login**:

POST http://localhost:8081/api/v1/auth/login Body: { "username": "admin", "password": "password" }
**2. Save tokens from response**
**3. Test protected endpoint**:

GET http://localhost:8081/api/v1/vehicles Header: Authorization: Bearer {accessToken}
**4. Test refresh**:

POST http://localhost:8081/api/v1/auth/refresh Body: { "refreshToken": "{refreshToken}" }

---

# Migration Summary

**Implementation Completed**: January 2026
**Security Features**:

- ✅ JWT-based authentication
- ✅ Role-based access control
- ✅ Permission-based authorization
- ✅ Token refresh mechanism
- ✅ Stateless session management
- ✅ CORS configuration
- ✅ Password encryption
- ✅ Method-level security

**Status**: Production Ready ✅

---

# Future Enhancements

## 1. Multi-Factor Authentication (MFA)

Implement TOTP-based 2FA for enhanced security.

## 2. OAuth2 Integration

Support login with Google, Facebook, etc.

### 3. Token Blacklist

Implement Redis-based token blacklist for immediate logout.

### 4. Audit Logging

Log all authentication attempts and security events.

### 5. Password Policies

Enforce password complexity, expiration, and history.

---

# Conclusion

The Spring Security and JWT implementation provides a robust, scalable, and secure authentication system. The role-based permission system offers fine-grained access control, while JWT tokens enable stateless authentication perfect for modern APIs.

---

**Developed by**: Maximo Andriola
**For**: Ricardo de ESEA SA
**Date**: January 2026
**Version**: 1.0
**Status**: Production Ready ✅